Ders 20 Çalışma Özeti

Ders 20 Çalışma Özeti

Genel Konular

- Dersin tarihi 9 Aralık 2020 Çarşamba. Bu, günün ikinci kaydıdır: uygulama (ödev) oturumu.
  - İki grubun öğrencileri aynı oturumda toplandı.
  - Konu anlatımı yoktur. Oturum ikinci ödevin kontrolü ve çözümüyle geçer.
- **Ödev kontrolü (yaklaşık 00:05 - 01:16):** Asistanlar, listede adı olan öğrencileri ayrı odalara alıp ödevlerini kontrol etti.
  - Kaydın bu kısmı odaların ayarlanması ve ses sorunlarıyla ilgili konuşmalardır.
- **Ödev çözümü (yaklaşık 01:16 - 01:37):** Bir asistan, ikinci ödevin sözde kodunu (pseudocode) anlattı.
  - Konu: dosyalardaki kelimeleri, geçtikleri dosya adlarıyla birlikte bir hash tablosunda saklamak.
  - Çakışma (collision) çözümü için çift hash (double hashing).
  - Kelimeden anahtar (key) üretmek için Horner yöntemi.
  - Doluluk oranı (load factor) kontrolü.
  - Ekleme (insertion) ve arama (search) adımları.

Hocanın Özellikle Vurguladığı Kısımlar

- Asistanın vurguları:
  - Çakışma bir hata değildir. Çift hash'te çakışma beklenir ve bir sonraki adımda başka bir indekse bakılır.
  - Arama, eklemeyle neredeyse aynı adımları izler: önce kelimenin olması gereken ilk yere, sonra ikinci yere bakılır.
  - Kelimeler büyük-küçük harf duyarsız (case insensitive) olmalı. Önce hepsi küçük harfe çevrilir.
  - Dosya adlarını tutmak için bağlı liste (linked list) en uygun yapıdır. Dinamik ya da statik dizi kullanan ödevler de doğru kabul edildi.

Kısa Tekrar Notları

- Tablo boyutu $M_{h1} = 997$ (1000'e en yakın asal), ikinci boyut $M_{h2} = 996$.
- Her tablo gözü bir öğe (item) tutar: indeks, kelime (value) ve kelimenin geçtiği dosya adları.
- Kelime küçük harfe çevrilir, Horner yöntemiyle sayısal anahtara dönüştürülür.
- İlk indeks $h_1(key)$ ile, adım boyu (step size) $h_2(key)$ ile hesaplanır.
- Çift hash: i. denemede indeks $(h_1(key) + i \cdot h_2(key))$ mod M_{h1}.
- Eklemede döngüden çıkış nedenleri:
  - Göz boş: öğe yazılır, doluluk oranı $1/M_{h1}$ kadar artar.
  - Gözde aynı kelime var: yalnız dosya adı eklenir.
  - Deneme sayısı tablo boyutuna ulaştı: tablo dolu.
- Doluluk oranı < 0.8 ise normal ekleme, 0.8 ile 1.0 arasında uyarıyla ekleme, ≥ 1.0 ise hata.
- Aramada boş göze rastlanırsa kelime tabloda yoktur.

Detaylı Açıklamalar

Ödevin amacı

- Elde bir dizi dosya ve bu dosyalarda geçen kelimeler var.
- Bir kelimenin hangi dosyalarda geçtiğini bulmak istenir. Doğrudan dosyalarda aramak uzun sürer.
- Kelimeler bir hash tablosunda indekslenirse arama ortalama sabit zamanda yapılır.
- Daha önce oluşturulmuş bir tablo dosyaya kaydedildiyse, program onu ve son doluluk oranını dosyadan okur.

Ödev sözde kodunun başı

Asistanın gösterdiği PDF'den (kareden okundu):

```
Mh1 <- 997
Mh2 <- 996
item <- (item.index, item.value, item.fileName[MAXNUMBEROFFILE])
allocate hashTable[sizeof(item) * Mh1]

if read Stored Hash Table from file
    hashTable  <- oldHashTable
    LoadFactor <- lastLoadFactor

#INSERTION
while read word from file and LoadFactor < 0.8:
    LoadFactor <- insertHashTable(hashTable, word, fileName, LoadFactor, Mh1, Mh2)

while read word from file and LoadFactor >= 0.8 and LoadFactor < 1.0:
    LoadFactor <- insertHashTable(hashTable, word, fileName, LoadFactor, Mh1, Mh2)
    display warning and current load factor

if LoadFactor >= 1.0
    display error
```

Ekleme fonksiyonu

1. Kelime küçük harfe çevrilir ve yeni öğenin değeri olarak yazılır.
2. Horner yöntemiyle kelimeden bir anahtar hesaplanır.
3. Anahtar h_1'den geçirilir. İlk aday indeks budur. Dosya adı öğeye eklenir.
4. Çakışma varsa (göz dolu ve içindeki kelime farklı) ve deneme sayısı tablo boyutundan küçükse:
   - h_2 ile adım boyu hesaplanır.
   - Yeni indeks hesaplanır, deneme sayacı (rotation) bir artırılır.
   - Yeni göz yeniden kontrol edilir.
5. Döngüden çıkınca nedene bakılır:
   - **Göz boş:** Öğe yazılır. Doluluk oranı $1/M_{h1}$ artar.
   - **Aynı kelime var:** Kelime daha önce eklenmiş. Yalnız dosya adı listeye eklenir.
   - **Deneme sayısı doldu:** Bütün gözlere bakılmış demektir. Tablo dolu mesajı verilir. Üzerine yazma (overwrite) istenmiyordu.
6. Fonksiyon güncel doluluk oranını döndürür.

Asistanın PDF'indeki çakışma döngüsü (kareden okundu):

```
#COLLISION VARSA
while hashTable[newItem.index] != NULL and
      strcmp(hashTable[newItem.index].value, newItem.value) != 0 and
      rotation < Mh1:
    stepSize <- hashFunction2(key, Mh2)
    newItem.index <- hashFunction(h1, stepSize, rotation, Mh1)
    rotation++
```

```
if hashTable[newItem.index] == NULL:          #yeri boş
    hashTable[newItem.index] <- newItem
    LoadFactor <- LoadFactor + (1/Mh1)
else if strcmp(hashTable[newItem.index].value, newItem.value) == 0:   #önceden kayıtlarda mevc
    append fileName hashTable[newItem.index].fileName[]
```

Arama fonksiyonu

- Kelime küçük harfe çevrilir, Horner ile anahtar, h_1 ile ilk indeks hesaplanır.
- Göz dolu ve içindeki kelime aranan değilse, çift hash ile bir sonraki aday göze geçilir.
- Döngüden çıkış:
  - **Göz boş:** Kelime tabloda yoktur.
  - **Deneme sayısı doldu:** Bütün tablo tarandı, kelime bulunamadı. Bu en kötü durumdur.
  - **Kelime eşleşti:** Kelime bulundu. Geçtiği dosyalar yazdırılır.

Öğrenci soruları

- **Çakışmada hata verilmeli mi?** Hayır. Çakışma beklenen bir durumdur. Bir sonraki adımda başka bir göze bakılır.
- **Göz NULL ise döngüye girilir mi?** Hayır. Döngü koşulları VE (and) ile bağlıdır. Biri sağlanmazsa döngüye girilmez. Döngüden sonra çıkış nedenine bakılır.
- **Tablo dosyaya nasıl kaydedildi?** Öğrencilerin çoğu metin (text) dosyası kullandı. Her satıra tablonun bir gözü yazıldı.
  - Bazıları 997 satırın hepsini boş satırlarla birlikte yazdı. Bazıları yalnız dolu gözleri, başlarında indeks numarasıyla yazdı.
  - Asistan, yapıların (struct) ikili (binary) dosyaya blok blok yazılıp okunabileceğini de belirtti.

Ders sonrası notlar

- **Asal tablo boyutu:** 997, 1000'den küçük en büyük asaldır ve 1000'e en yakın asaldır (1009'dan daha yakın). Tablo boyutunun asal olması, çift hash'te her adım boyunun bütün gözleri dolaşmasını sağlar.
- **İkinci hash fonksiyonu:** h_2 hiçbir zaman 0 vermemelidir. Yoksa aynı göze bakılıp durulur. Yaygın seçim büyük olasılıkla $h_2(key) = 1 + (key \bmod M_{h2})$'dir. $M_{h2} = 996$ ile bu, 1 ile 996 arasında bir adım verir.
- **Deneme sayacının başlangıcı:** PDF'deki döngüde yeni indeks hesaplandıktan sonra sayaç artırılıyor. Sayaç 0'dan başlıyorsa ilk çakışmada aynı göz bir kez daha hesaplanır. Sayacı 1'den başlatmak bu gereksiz adımı önler.
- **Horner yöntemi ve taşma:** Uzun kelimelerde $\sum c_k R^{L-k-1}$ çok büyür ve tam sayı taşar. Her adımda mod alınarak hesaplanması önerilir.
- **Boş gözde aramayı bitirmek:** Bu kural yalnız tablodan silme yapılmıyorsa doğrudur. Silme olsaydı, silinen gözler "silindi" (tombstone) diye işaretlenmeliydi.
- Altyazıda "H tablosu", "haç tablosu" diye geçen terim hash tablosu, "Sudo kodu" diye geçen terim sözde koddur.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.